

PhishGuard — Complete Architecture & Implementation Plan

1. Introduction

PhishGuard is a phishing detection platform built around a browser extension that analyzes URLs, webpages, domains, and browser activity in real time to detect phishing attacks before users submit credentials.

The system combines:

- Browser-side analysis
- URL feature engineering
- DNS and WHOIS enrichment
- Threat intelligence
- Machine learning classification
- SOC-style incident workflows
- Analytics and monitoring

The goal is not only to classify a URL as malicious or benign, but also to:

- explain WHY the decision was made,
- support incident response,
- continuously improve detection quality,
- and reduce false positives.

The architecture is designed to be modular so that heuristic systems can initially power detection while machine learning models are added later.

2. Core System Objectives

The system should:

1. Detect phishing URLs before credential submission
 2. Analyze webpages in real time
 3. Detect typosquatting and domain impersonation
 4. Perform live DNS and WHOIS enrichment
 5. Use threat intelligence feeds
 6. Generate explainable risk scores
 7. Maintain SOC investigation queues
 8. Provide analytics dashboards
 9. Continuously retrain ML models
 10. Minimize browser-side computational overhead
-

3. Complete Detection Workflow

This section explains exactly how a phishing URL moves through the system.

Step 1 — Browser Navigation Monitoring

The extension continuously monitors:

- tab changes,
- navigation events,
- redirects,
- URL changes,
- form submissions.

When a user opens a webpage:

1. The extension captures the URL.
2. The URL is normalized.
3. Features are extracted locally.
4. The data is sent to the backend.

Example:

`https://paypal-login-secure-update.com/verify/account`

The extension should immediately separate:

- protocol,
 - hostname,
 - subdomains,
 - TLD,
 - path,
 - query parameters.
-

4. URL Feature Engineering

URL features are the most important component of the initial MVP.

Feature engineering means converting URLs into measurable signals that a model can understand.

4.1 Length Features

Phishing URLs are often unusually long because attackers:

- hide malicious paths,
- mimic trusted brands,
- add redirect parameters,
- include random tokens.

Features

Feature	Description
url_length	Total URL length
hostname_length	Domain length
path_length	URL path length
query_length	Query parameter length

Example:

```
https://paypal.verify.account.security-update.ru/login/verify/user/account
```

Long URLs are suspicious but not automatically malicious.

4.2 Lexical Features

Lexical features analyze the text composition of the URL.

Important Features

Feature	Purpose
dot_count	Many subdomains often indicate phishing
hyphen_count	Excessive hyphens are suspicious
digit_count	Random digits may indicate automation
special_char_count	Obfuscation indicator
suspicious_keywords	login, verify, secure, update
entropy_score	Detects random-looking domains

Example:

```
secure-login-update-paypal-authentication.com
```

High keyword density increases suspicion.

4.3 Typosquatting Features

Typosquatting is when attackers create domains similar to legitimate brands.

Examples:

```
paypa1.com  
faceb00k.com  
google-auth-login.net
```

Levenshtein Distance

Measures how many edits are required to transform one string into another.

Example:

```
paypal -> paypa1  
Distance = 1
```

Small edit distance from major brands is highly suspicious.

Homoglyph Detection

Attackers replace characters with visually similar ones.

Example:

```
rnicrosoft.com
```

Looks visually similar to:

```
microsoft.com
```

Unicode normalization should be performed before analysis.

4.4 Domain Features

Important Domain Features

Feature	Purpose
tld_type	Suspicious TLDs
subdomain_depth	Excessive nesting
ip_based_url	Raw IP instead of domain
https_usage	HTTPS presence
certificate_age	New certificates are suspicious

4.5 DNS Features

The backend performs live DNS lookups.

DNS Information Collected

Data	Purpose
A record	Maps domain to IP
MX record	Mail infrastructure
NS record	Nameservers
TTL	DNS behavior
ASN	Hosting provider reputation

Fast Flux Detection

Fast flux is when attackers rapidly rotate IP addresses.

Indicators:

- rapidly changing DNS records,
- multiple unrelated IPs,
- abnormal TTL values.

This helps attackers evade blacklists.

4.6 WHOIS Features

WHOIS/RDDS enrichment provides registration metadata.

Important Features

Feature	Purpose
domain_age_days	New domains are suspicious
registrar	Some registrars abused heavily
registration_length	Short registrations suspicious
nameserver_patterns	Threat infrastructure reuse

A domain registered within the last few days should significantly increase risk.

4.7 Redirect Features

Phishing pages frequently use redirects.

Example:

bit.ly -> redirect site -> phishing page

Features

Feature	Purpose
redirect_count	Excessive redirecting
shortening_service	URL concealment
cross_domain_redirects	Infrastructure hopping

5. Page Content Analysis

After page load, the content script analyzes webpage structure.

This is important because many phishing pages may appear legitimate at URL level.

5.1 HTML Features

Login Form Detection

Detect:

```
<input type="password">
```

Credential collection forms are strong phishing indicators.

Hidden Elements

Detect:

- hidden iframes,
 - invisible overlays,
 - obfuscated scripts,
 - malicious JavaScript.
-

External Resource Analysis

Phishing pages often:

- copy images from real brands,
- load scripts from unrelated domains,
- mix legitimate and malicious resources.

Features:

Feature	Purpose
external_script_ratio	Suspicious dependencies
iframe_count	Hidden embedded content
script_entropy	Obfuscated JavaScript

6. Threat Intelligence Layer

Threat intelligence enriches detections using external reputation systems.

6.1 Threat Intelligence Sources

External Feeds

Source	Purpose
VirusTotal	Multi-engine reputation
PhishTank	Community phishing DB

Source	Purpose
OpenPhish	Live phishing URLs
URLHaus	Malware URLs

6.2 Threat Intelligence Workflow

The backend checks:

- URL reputation,
- IP reputation,
- ASN reputation,
- historical detections,
- malware associations.

This creates:

Threat Intelligence Score

Threat intelligence should NOT be the only detection mechanism because:

- new phishing domains may not exist in databases,
- attackers rotate infrastructure rapidly.

7. Machine Learning Architecture

The ML model receives structured features extracted from:

- URLs,
- domains,
- DNS,
- WHOIS,
- webpage content,
- threat intelligence.

7.1 Recommended Initial Models

Start with classical ML.

Recommended Models

Model	Reason
Random Forest	Strong baseline
XGBoost	Excellent structured-data performance
Logistic Regression	Explainability
LightGBM	Scalable boosting

Deep learning should come later.

7.2 Training Dataset

Benign URLs

Collect from:

- Common Crawl
- Alexa/Tranco domains
- real browsing traffic

Malicious URLs

Collect from:

- PhishTank
 - OpenPhish
 - URLHaus
 - VirusTotal
-

7.3 ML Pipeline

```
Dataset Collection
  ↓
Feature Extraction
  ↓
Cleaning & Normalization
  ↓
Train/Test Split
  ↓
Model Training
  ↓
Validation
  ↓
Feature Importance Analysis
```

↓
Deployment

7.4 Explainability

The system must explain detections.

Example:

```
Risk Score: 91
Reasons:
- Newly registered domain
- Login form detected
- Typosquatting similarity to PayPal
- Known malicious ASN
```

SOC analysts must understand WHY the system flagged a page.

8. Decision Engine

The final detection score combines multiple layers.

8.1 Scoring Formula

Example:

```
Final Score =
0.35 * ML Score +
0.25 * Threat Intelligence +
0.20 * URL Heuristics +
0.20 * HTML Analysis
```

8.2 Severity Levels

Severity	Action
Low	Allow
Medium	Warning
High	Block

Severity	Action
Critical	Block + SOC alert

9. Browser Extension Architecture

The extension should be lightweight.

Heavy processing should occur in the backend.

9.1 Extension Components

Background Service Worker

Responsible for:

- URL monitoring,
 - API communication,
 - event handling,
 - caching.
-

Content Script

Responsible for:

- webpage inspection,
 - DOM analysis,
 - form detection,
 - HTML feature extraction.
-

Popup UI

Displays:

- risk score,
 - detection reasons,
 - domain information,
 - report button.
-

Local Cache

Used for:

- reducing repeated lookups,
- storing recent detections,
- offline heuristics.

10. Backend Architecture

The backend acts as the central analysis system.

10.1 Backend Services

Service	Responsibility
API Gateway	Request routing
Feature Service	Feature extraction
Threat Intel Service	Reputation lookups
ML Service	Classification
Incident Service	SOC workflow
Analytics Service	Metrics and dashboards
Auth Service	Authentication

10.2 Suggested Backend Stack

Component	Suggested Technology
API	FastAPI
ML Service	Python + Scikit-learn
Database	PostgreSQL
Cache	Redis
Queue	RabbitMQ/Kafka
Dashboard Backend	Node.js/FastAPI

11. SOC Workflow System

SOC functionality transforms PhishGuard from a browser tool into a security platform.

11.1 Incident Creation

Critical detections create incidents.

Stored information:

- URL,
 - domain,
 - timestamp,
 - user,
 - IP,
 - threat score,
 - detection reasons,
 - screenshots,
 - WHOIS data.
-

11.2 Analyst Queue

Analysts can:

- review alerts,
 - mark false positives,
 - escalate incidents,
 - block domains,
 - add indicators to blacklist,
 - close incidents.
-

11.3 Feedback Loop

Analyst decisions feed back into:

- reputation database,
- ML retraining datasets,
- heuristic tuning.

This continuously improves accuracy.

12. Analytics Dashboard

The analytics dashboard visualizes system activity.

12.1 Dashboard Features

Threat Metrics

- total detections,
- phishing trends,
- blocked domains,
- attack categories.

SOC Metrics

- analyst workload,
- false positive rate,
- incident severity,
- mean response time.

Threat Intelligence Metrics

- top malicious ASNs,
- most abused TLDs,
- active campaigns.

12.2 Suggested Dashboard Stack

Component	Technology
Frontend	React
Charts	Recharts/Chart.js
Backend	FastAPI
Auth	JWT/OAuth

13. Recommended Project File Structure

```
phishguard/  
├──  
└─ extension/
```

```
|   |─ manifest.json
|   |─ background/
|   |─ content/
|   |─ popup/
|   |─ assets/
|   |─ utils/
|
|─ backend/
|   |─ api/
|   |─ services/
|   |─ ml/
|   |─ threat_intel/
|   |─ database/
|   |─ queues/
|   |─ auth/
|   |─ analytics/
|
|─ dashboard/
|   |─ src/
|   |─ components/
|   |─ charts/
|   |─ pages/
|
|─ datasets/
|
|─ notebooks/
|
|─ scripts/
|
|─ docker/
|
|─ docs/
```

14. Execution Plan

The project should be built in phases.

Phase 1 — Browser Extension MVP

Build:

- URL monitoring,
- heuristic engine,
- blacklist checking,
- popup warnings.

No ML initially.

Goal:

Create a working phishing detector quickly.

Phase 2 — Backend Services

Build:

- FastAPI backend,
 - DNS lookups,
 - WHOIS enrichment,
 - threat intelligence APIs.
-

Phase 3 — Machine Learning

Build:

- feature extraction pipeline,
- datasets,
- model training,
- evaluation pipeline.

Deploy:

- Random Forest,
 - XGBoost.
-

Phase 4 — SOC Queue

Build:

- incident tracking,
 - analyst workflows,
 - threat management.
-

Phase 5 — Analytics Dashboard

Build:

- visual analytics,
- campaign tracking,
- SOC metrics,

- reporting.

15. Suggested Technology Stack

Layer	Technology
Extension	JavaScript/TypeScript
Backend API	FastAPI
ML	Python + Scikit-learn
Queue	RabbitMQ/Kafka
Database	PostgreSQL
Cache	Redis
Dashboard	React
Containerization	Docker
Deployment	Kubernetes (later)

16. How to Execute the System

Backend

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload
```

Extension

1. Open Chrome
2. Go to:

```
chrome://extensions/
```

1. Enable Developer Mode
2. Load unpacked extension
3. Select:

```
extension/
```

Dashboard

```
cd dashboard  
npm install  
npm run dev
```

17. Long-Term Improvements

Future improvements:

- transformer-based phishing detection,
 - screenshot similarity analysis,
 - OCR-based brand impersonation detection,
 - graph-based infrastructure analysis,
 - real-time collaborative threat sharing,
 - browser isolation,
 - autonomous incident response.
-

18. Final Engineering Notes

Phishing detection is fundamentally a probabilistic security problem.

No single indicator proves maliciousness.

A strong system combines:

- heuristics,
- machine learning,
- threat intelligence,
- browser analysis,
- authentication signals,
- and human analyst feedback.

The biggest engineering challenge is balancing:

- detection rate,
- latency,
- explainability,
- and false positives.

For an MVP, prioritize:

1. reliable feature extraction,
2. fast URL analysis,
3. explainable scoring,
4. stable backend architecture.

Do NOT begin with deep learning.

Most successful phishing systems initially succeed because of:

- good feature engineering,
- strong heuristics,
- quality enrichment,
- and feedback loops.

Machine learning improves the system later, but architecture and data quality determine whether the platform succeeds.